

Tema 6: Detección de colisiones

Programación Gráfica

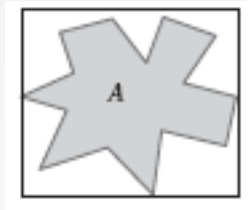
Marcos Novalbos Mendiguchía
Marcos.Novalbos@live.u-tad.com

Introducción

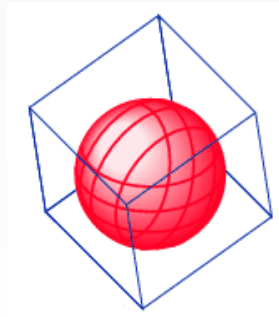
- Sistemas comunes en detección de colisiones:
 - Usar una representación simple de un objeto complejo (envolverlo)
 - Crear una jerarquía de objetos sencillos que forman la escena (árboles o subdivisión espacial)
 - Cuando se encuentran dos objetos sencillos que colisionan, se pasa al grano fino (test a nivel de polígonos)

1.1: Volúmenes envolventes

- En 2D se representarían como polígonos que albergan en su interior nuestra figura:



- En 3D tenemos figuras volumétricas:



1.1: Características deseables

- Las propiedades para elegir los volúmenes envolventes óptimos son:
 - Test de intersección muy rápidos
 - Entorno “ajustado” al objeto
 - Rápidos de crear/calcular
 - Fáciles de transformar y rotar
 - Coste bajo de memoria

1.1: Características deseadas

- Se necesita buscar el equilibrio entre las características deseadas:
 - Cuanto más apurado sea el volumen envolvente, consumirá más memoria y el test será más lento, pero tendrá mejores resultados.

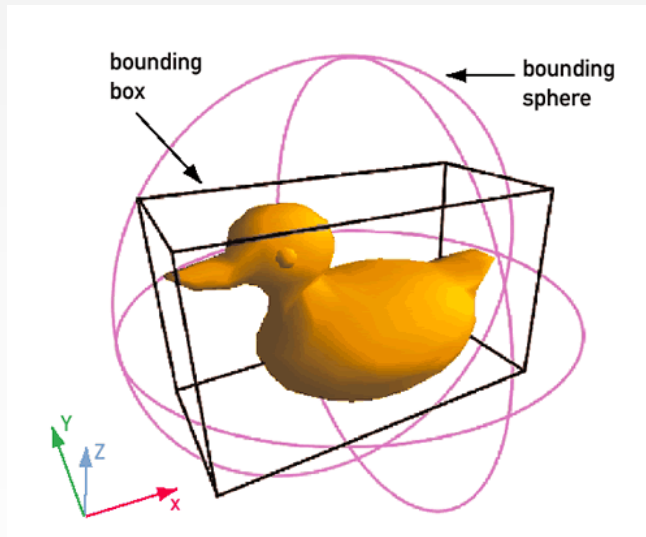


1.1: Esferas

- Las esferas son el BV más sencillo que se puede construir. Cumple:
 - Rápido de construir
 - Bajo consumo de memoria: Se representan con el centro y el radio
 - Test de intersección muy rápidos.
- No cumple:
 - Tests de intersección muy poco precisos

1.1: Axis Aligned Bounding Boxes

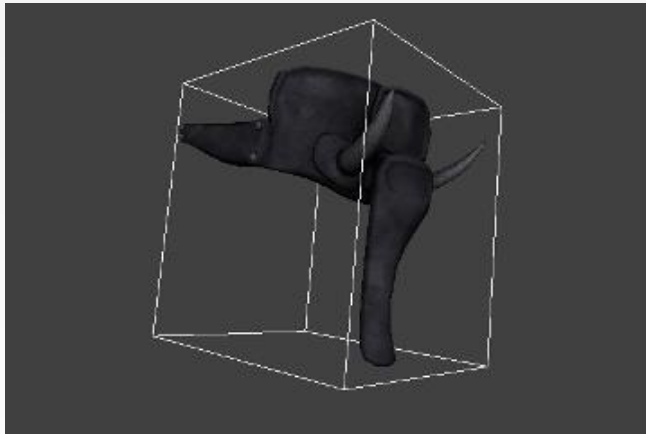
- Los AABB se representan como exaedros que tienen sus caras alineadas con los ejes x, y, z



- Ventajas:
 - Mejora la precisión de los tests
 - Fácil de construir
 - Consume poca memoria
- Inconvenientes:
 - No ajusta igual en todos los casos
 - Si el objeto cambia su forma u orientación, hay que recalcularlo

1.1: Oriented Bounding Boxes

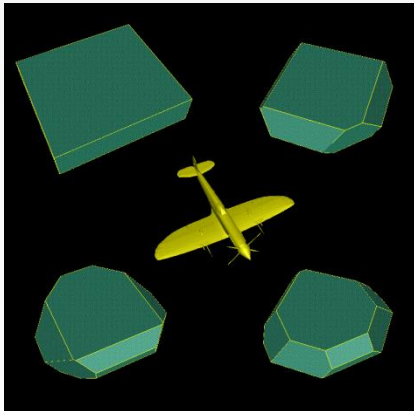
- Se mantiene la idea de usar exaedros como volúmenes envolventes, pero ahora no tienen que estar alineados con los ejes.



- Ventajas:
 - No hay necesidad de recalcular
 - El mejor ajuste posible (con cajas) en todos los casos
- Inconvenientes:
 - Más difícil de representar (más memoria)
 - Tests más complejos

1.1: Discrete-Orientation Polytopes

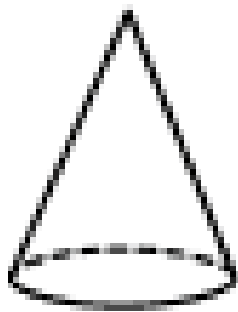
- Los DOPs son poliedros que tienen como característica que sus caras son paralelas dos a dos. Son una generalización de las cajas envolventes, y se pueden construir a partir de cajas OBBs más pequeños:



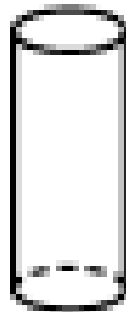
- Ventajas:
 - Test más precisos
 - Poco incremento de memoria
- Inconvenientes
 - Tests más complejos
 - Construcción más compleja

1.1: Otros volúmenes envolventes

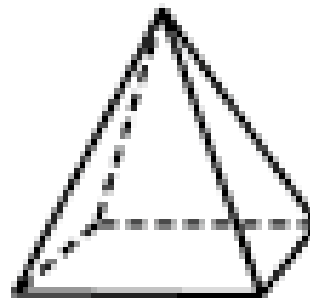
- Cilindros, conos, elipsoides, pirámides, otros poliedros... En general cualquier otro tipo de volumen envolvente tendrá un coste de creación o de realización de tests mucho más altos.



cone



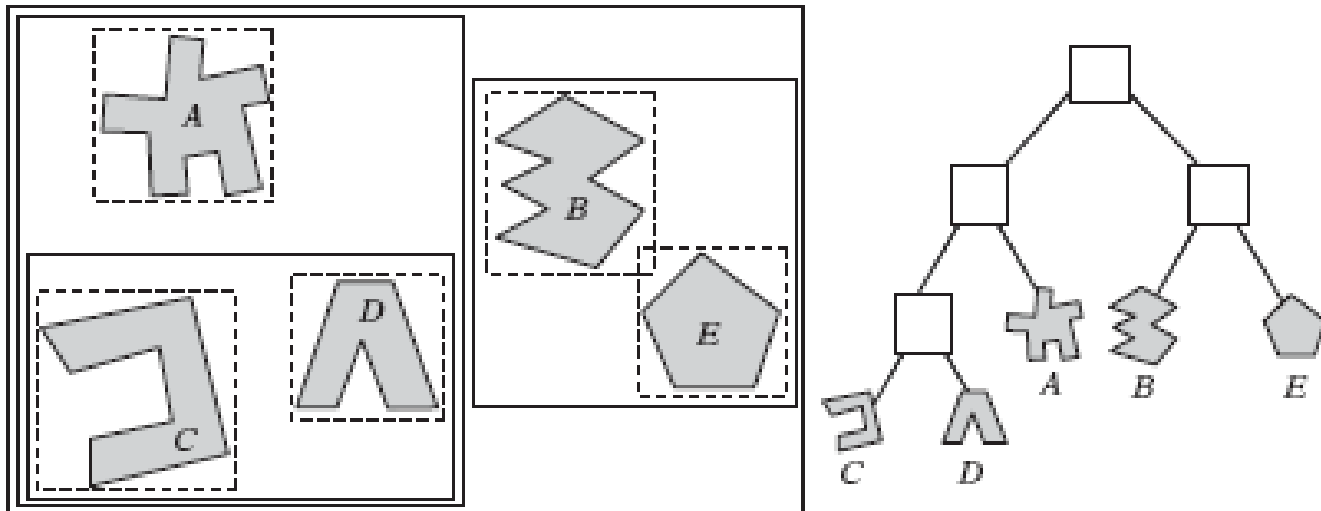
cylinder



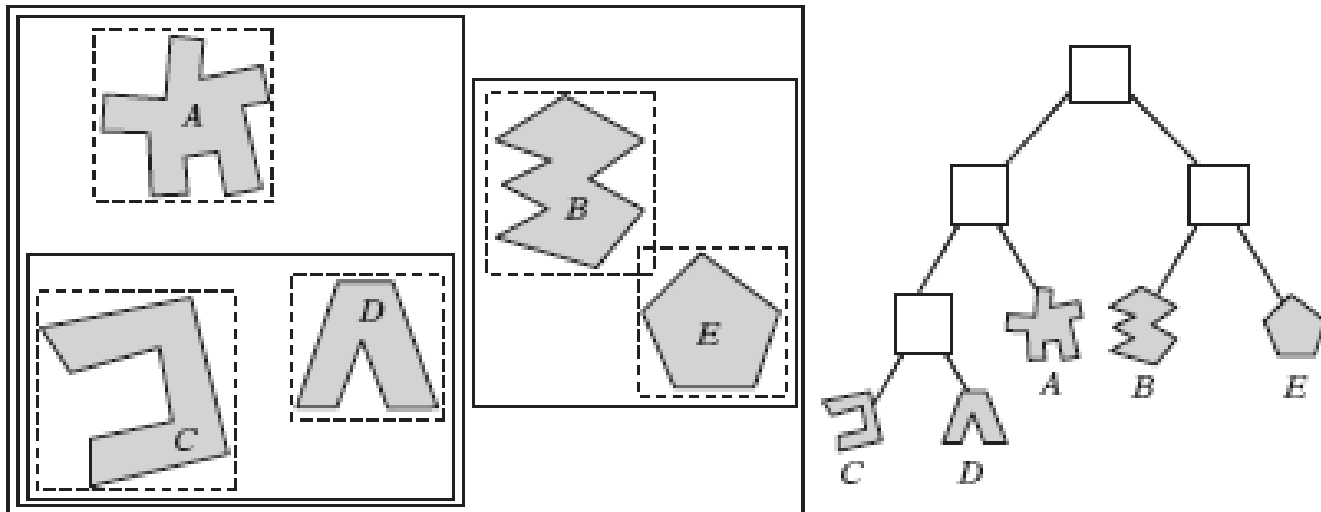
pyramid

Jerarquías de volúmenes envolventes

- Cuando se tiene un escenario con varios objetos que podrían estar colisionando, es necesario establecer algún tipo de jerarquía para los test.
- En este caso, se parte de volúmenes envolventes, que a su vez están contenidos en otros volúmenes envolventes mayores



- La jerarquía se representa en forma de árbol, de forma que los intermedios son BV contenedores de otros BV, y los nodos hoja son BV que contienen a los objetos únicamente.
- Se mantiene la relación de distancia de los objetos dentro del árbol.



1.2: Jerarquías de volúmenes envolventes

- Características:
 - Cada nodo de la jerarquía debe contener el mínimo volumen
 - Igualmente, la suma de todos los volúmenes debe ser mínima
 - Se debe de tener mucho cuidado con los nodos más cercanos a la raíz. (Poder cortar una rama desde ahí optimiza mucho los tests)
 - El volumen de solapamiento de nodos hermanos debe ser mínima.
 - La jerarquía debe estar balanceada con respecto a los nodos y su contenido

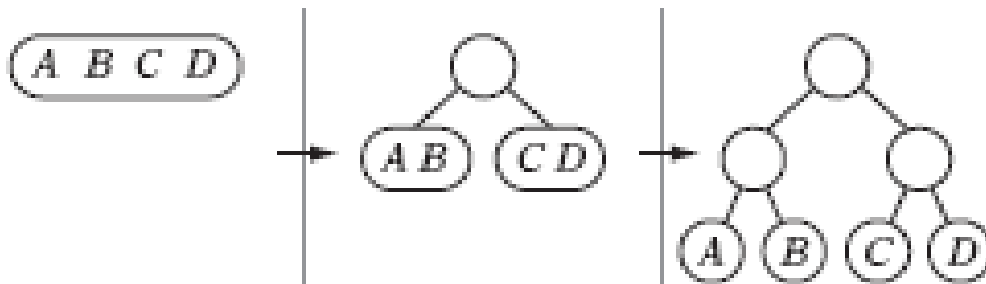
1.2: Contrucción

- La construcción de un árbol que cumpla las características anteriores es un proceso laborioso, que no siempre resulta en el árbol óptimo. Hay 3 estrategias:
 - Top-down
 - Bottom-up
 - Inserción

1.2: Top-down

- El método más usado. En este caso, se parte de un volumen envolvente que contenga a todos los objetos, y se va dividiendo en volúmenes más pequeños, a la vez que se divide el árbol:

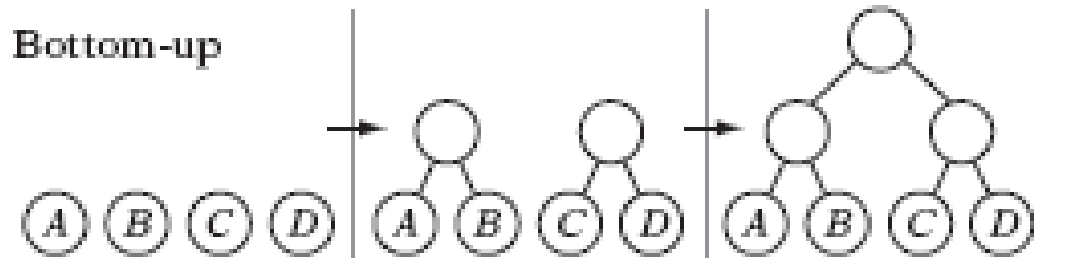
Top-down



- Ventajas:
 - Rápido
 - Fácil de implementar
- Inconvenientes:
 - No genera los mejores árboles

1.2: Bottom-Up

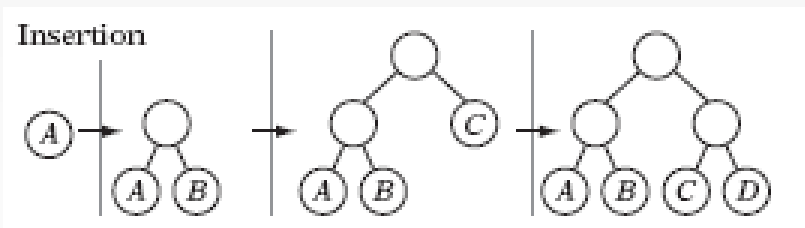
- En este caso, se parte de los volúmenes envolventes más básicos (los objetos), y se van construyendo volúmenes envolventes más grandes a partir de los más cercanos.



- Ventajas:
 - Mejores árboles
- Inconvenientes:
 - Más difícil de implementar
 - Más tiempo de cómputo

Inserción

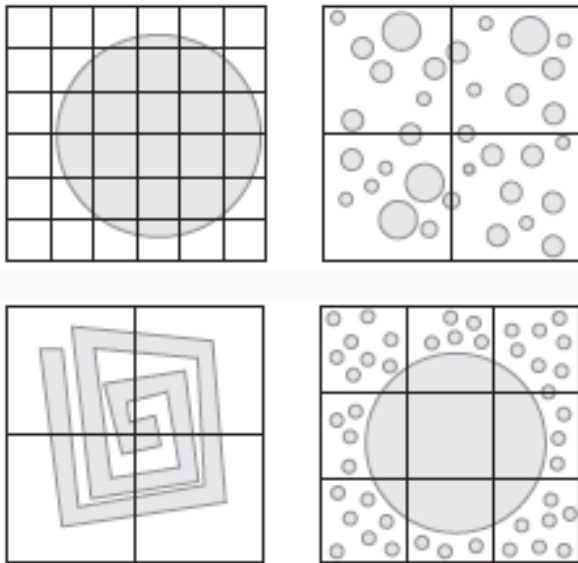
- Se empieza por un árbol vacío, y se van insertando los nodos de forma que se mantenga el equilibrio entre las ramas.



- Ventajas:
 - Rápido como top-down
 - Mejores resultados que bottom up

2: Spatial partition

- Este sistema se basa en la división del espacio en celdas de tamaños similares para después analizar los polígonos que caen dentro.



- Uno de los problemas más grandes que presenta es la elección de un tamaño de celda apropiado, pero suele dar buenos resultados.
- Lo ideal sería una división cercana al tamaño de los elementos de la escena, pero eso no es siempre posible

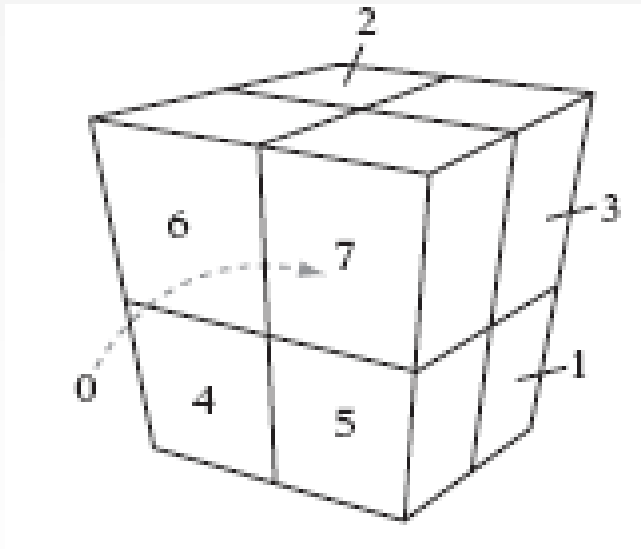
2: Grids jerárquicos (división espacial)

- Para dar una solución al problema anterior, se puede recurrir a una jerarquía de grids. De esta forma se empieza a particionar el espacio con grids de grano grueso, y en el momento que se necesite se subdivide una celda en celdas más pequeñas.
- A la hora de realizar los test de colisiones, se iría directamente a las subceldas más pequeñas

B						D					
										F	
A				C					E		

2: Árboles (división espacial)

- Otra forma de representar el espacio particionado es mediante árboles.
- Se representa con árboles en los que cada nodo es una celda del grid.



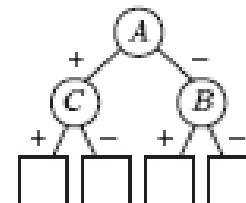
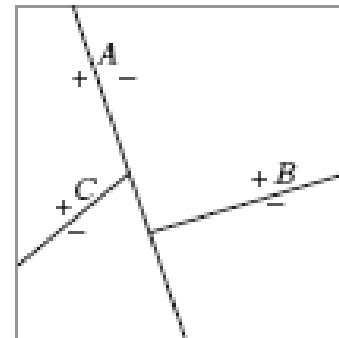
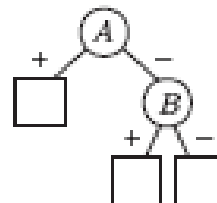
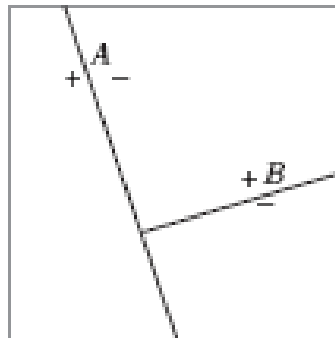
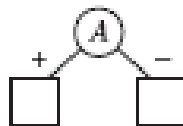
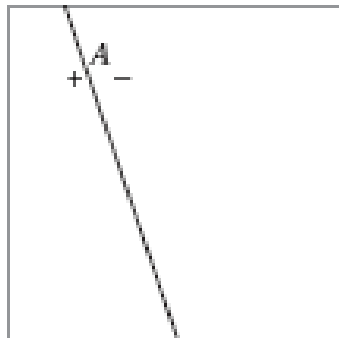
- Los nodos padre son celdas de grano grueso, y cuelgan “n” hijos (8 en el caso de los octrees, 4 en los quadrees)
- Los nodos hijo son el resultado de dividir el nodo padre en “n” divisiones iguales.

2: Árboles (división espacial)

- La diferencia fundamental con los grids jerárquicos es la forma de navegar a través de estas estructuras:
 - Desde la raíz, hay que recorrer toda la jerarquía del árbol.
 - Si hay nodos vacíos, se pueden eliminar

BSP-Trees (división espacial)

- Los BSP-trees son árboles binarios que representan las particiones del escenario. En este caso, las particiones se realizan a partir de divisiones arbitrarias.



BSP-trees (división espacial)

- Este tipo de árboles no se suelen usar para simulaciones con objetos en movimiento, ya que habría que reconstruir el árbol en cada paso y es algo costoso.
- Se suelen usar para objetos que se encuentren en background y que rara vez se moverán.